模型训练中的递归自我改进

外部锚点的迁移与它的证据边界

61 篇编码、53 条来源、63 条摘要级证据：五层对象、四次锚点内移与六类失效

2026-11-05

摘要

「递归自我改进」（RSI）在过去四年从一句愿景变成了一批可运行的系统：模型自己出题、自己解题、自己当裁判、自己改自己的代码，甚至自己改进「改进的方法」。本文对这 61 篇工作做系统梳理，要回答的不是「能不能做到」，而是**这些闭环究竟在改什么，支撑它们的证据有多强**。

梳理得到一个与直觉不同的判断：真正的变化不在算法，而在**判断权的位置**。按「改什么」可分五层——上下文、外壳、评判器、权重、改进过程——而层与层之间最关键的区别不是效果大小，是**单次改动能否被验证、失败能否被回滚**。这两个性质同向变化：越往下，越不可验证、越难回滚。

在这个分层下，四年演进的形状变得清楚：外部锚点沿「人写答案 → 可判定答案 → 模型判断 → 统计量」单向内移。27 篇改权重的工作里，以可判定答案为奖励的 20 篇、以模型判断为奖励的 5 篇、以统计量替代判定的 3 篇。每内移一步都换来更大的可扩展性：从需要人工推理，到只需要一个测试，到只需要一个通过/不通过，最后到只需要一个标量。但每一步也都换来一类新的失效——判断者与被判断者同源，带来自我确认循环与判据饱和；用置信度代替对错，带来与目标的脱钩；改外壳带来验收门的失明，因为门只看得见分数，看不见自己正在拆掉第几层防护。

证据强度的落差比失效模式更值得警惕。对 90 篇自称 RSI 的研究，只有 22% 审计了两次以上的完整循环。而唯一能回答「能力边界有没有移动」的指标 pass@k，在本文编码的 61 篇里只有 2 篇报告——被最少报告的指标，恰好是最难看的那个。

关键词：递归自我改进；自举；可验证奖励；自我确认循环；模型坍缩；评测审计

1 引言

1.1 一个词，五种东西

「自我改进」在文献里至少有五种互不兼容的用法。它可能指模型改自己的提示，也可能指改自己的代码，也可能指改自己的权重、自己的评判器，甚至自己改进的方法。

这个领域用同一套词汇（self-refine、self-reward、self-play、self-evolve）覆盖了根本不同的志向 `[c-rsi-vocabulary]`。后果是可比较性丧失——两篇都叫 self-improving 的论文，可能一篇的失败可以删一行记录撤销，另一篇的失败只能重训。

本文的第一个任务就是把这个词拆开。拆分需要一个操作性的判据：**RSI 与自改进的区别不在效果大小，而在闭环的闭合度——被改进的对象是否包含「改进机制本身」** `[c-rsi-definition]`。DGM 改的是「自己改代码的能力」，是这一判据的最小实例；STaR 改的是模型参数，而改进机制（人选的循环）没有被改。

1.2 三个问题

- **Q1**：这些闭环在改什么？能否拆成可比较的层次？
- **Q2**：奖励信号从哪里来？它在往哪个方向移动？
- **Q3**：这些工作的证据，能不能支撑它们使用的词？

第三个问题最不讨喜，但它是本文的重点。前两个问题已有综述在回答，第三个问题的答案直到 2026 年才第一次被系统测量。

1.3 贡献

1. 给出一个可操作的五层分层，并指出**可验证性与回滚成本同向**，据此可以直接推出「哪一层可以放开自动改」。
2. 把奖励来源编码成可复算的分布，指出锚点内移不是比喻而是一个可统计的趋势，并说明每一步内移换来了什么、失去了什么。
3. 把「证据强度」当作独立的研究对象：不问这些工作对不对，而问它们提供的证据能不能支撑它们使用的词。

2 方法：语料与编码

2.1 检索与纳入

检索窗口 2022-01 至 2026-11，用 16 组 arXiv 检索式取并集，得到候选池 1006 篇。按「提出或测量一个自举闭环」纳入 61 篇（2022-03 至 2026-11），编码为三个维度：改什么（layer）、奖励从哪来（reward）、摘要报什么指标（metric）。全部编码写

在 corpus_labels.csv，统计由 analyze.py 重算。

纳入的 61 篇按年份分布为 2022 年 1 篇、2023 年 9 篇、2024 年 14 篇、2025 年 22 篇、2026 年 15 篇 [c-corpus-timeline]（图 1）。2026 年只统计到 11-05，这个数不应读作降温。

一处必须声明的偏差：首轮凭记忆列出的 66 个经典工作 arXiv ID 里，有 11 个是错的——标题配到了别的论文上。全部经 id_list 逐条核对后剔除，或改由标题检索补回。这件事的教训写在 §10.2：**校验器检查引用是否闭合，不检查引用是否指向你以为的那篇论文。**

2.2 证据链的三级结构

本文的每条判断都可回溯到三级：正文的方括号标记（形如 [c-anchor-migration]）指向一条**主张**，主张指向**证据**（arXiv 摘要的原文片段），证据指向**来源**（53 条 arXiv 预印本）。本文引用的 48 条主张、63 条证据，全部记录在归档的 CSV 里。

两条纪律值得单独说明。第一，所有 63 条引文都是从摘要中原样摘取的子串，写入前逐条校验，不做近似匹配。第二，凡涉及计数的数字（27 篇权重层、20 篇答案奖励等），都在主张里写明可重算的推导式，由审计脚本从语料重新数一遍——不是信任作者的数，而是逼它可复现。

3 分层：改的到底是什么

3.1 五层

按「改什么」分层是本档的操作性定义 [c-loop-layers]：

- **上下文层**：反思记录、记忆、技能库。模型不动，状态动。
- **外壳层**：代码、提示、工作流。模型不动，外围的代码动。
- **评判器层**：judge、reward model、verifier。判断标准本身被改进。
- **权重层**：模型参数。
- **过程层**：改进机制本身——即「以后怎么改」这件事被改。

分层本身不是新东西——自演化 agent 的综述早就按「模型、记忆、工具、架构」分过。两篇综述给出的出发点也是同一个：模型本身是静态的，不会随任务改变内部参数；而推理时自改进把增益放在计算而不是参数上 [c-survey-framing]。这恰好解释了为什么「自我改进」能同时容纳两条互不兼容的路线。

本文这张表的用处不在分类，在**最后两列**：回滚成本与单次改动的可验证性。这两列同向。上下文层的改动可以删一条记录撤销，验收就是「同一模型在同一任务上是否更好」；权重层的改动要重训，而且一次更新无法归因到具体哪条数据。过程层最糟：改了改进机制之后，**此前所有的验证结论都要作废**，因为你验证的是旧的机制。

3.2 分布

61 篇的构成是：权重层 27 篇、外壳层 13 篇、分析类 15 篇、上下文层 4 篇、评判器层 1 篇、过程层 1 篇 [c-corpus-composition]（图 2）。

这个分布有一个容易被忽略的含义：**外壳层 13 篇与上下文层 4 篇加起来 17 篇，比权重层的 27 篇少，但它们的工程采用率远高于权重层。**原因是成本——改一个提示不需要 GPU 集群。综述界与工业界的注意力分配在这里是错位的。

4 外部锚点的迁移

4.1 一个可统计的趋势

本文把权重层 27 篇按「奖励从哪来」编码 [c-anchor-migration]（图 3）：

- **可判定答案**（answer）20 篇：答案、单元测试、执行器、环境反馈。
- **模型判断**（judge）5 篇：自评，或学到的评判器。
- **统计量替代**（proxy）3 篇：置信度、熵、末 token 概率。

这张图最容易被误读成「大家都不再用答案了」。事实相反：**四分之三的权重层工作仍在用可判定答案。**正确的说法是——当答案不可判定时，替代品按「judge → proxy」的顺序出现，而且这个替代是有代价的。

4.2 第一段：人写答案

STaR 的循环写得极清楚：生成推理；答错的题用正确答案回填提示重试；只保留最终答对的推理去微调；重复 [c-star-loop]。它的量级声明是与 30 倍大的模型微调可比 [c-star-scale]——注意可比对象是 CommensenseQA 上的微调基线，不是通用能力。

ReST 把同样的循环组织成离线批次 [c-rest-offline]。SPIN 引入自博弈，但判别基准仍是「人写的回答」[c-spin-anchor]：人不再写推理，但仍在定义什么算好。Quiet-STaR 把锚点从标准答案换成后续文本 [c-quiet-star-anchor]，锚点开始由语料自带。

这一段里量级最干净的一条来自 Iterative RPO：在 GSM8K 上把准确率从 55.6% 提到 81.6% [c-

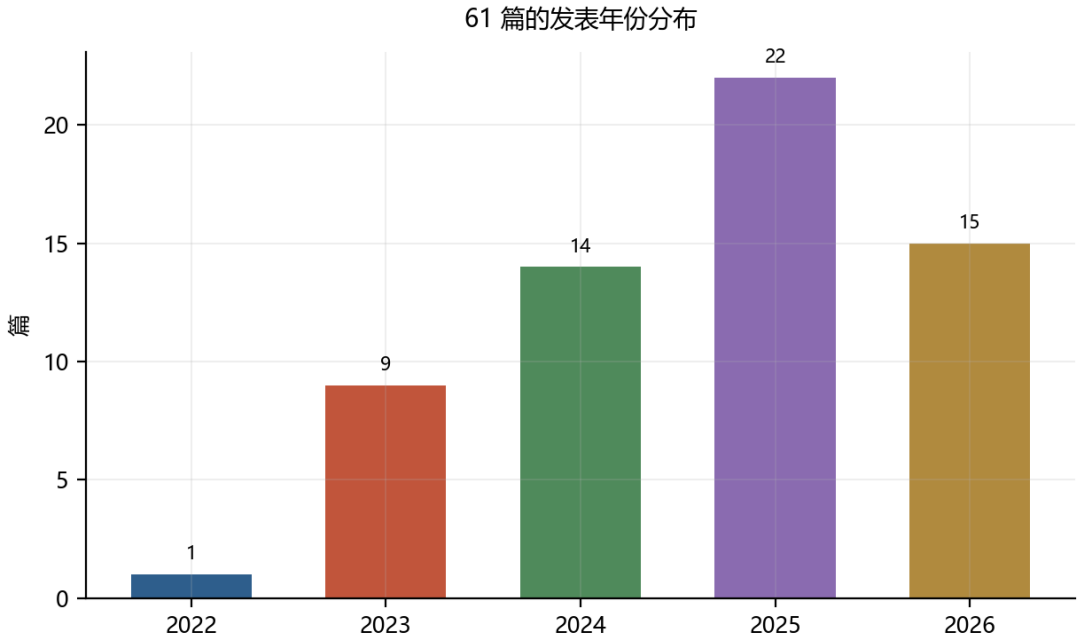


图 1　图 1　61 篇按发表年份分布。2025 年是峰值（22 篇）；2026 年只统计到 11-05，不应读作降温。

表 1　五层分层与它的两个性质。分层是本文的构造；回滚成本与可验证性同向，这是第 8 节建议的依据。

层	改的对象	回滚成本	单次改动的可验证性
上下文	反思、记忆、技能库	极低（删记录）	高（同模型同任务可比）
外壳	代码、提示、工作流	低（回退版本）	高但有陷阱（验收门会掩盖防护流失）
评判器	judge / reward model / verifier	中	中（评判器本身要被评判）
权重	模型参数	高（重训）	低（一次更新无法归因）
过程	改进机制本身	最高	最低（改后此前验证全部失效）

iterative-preference]。注意它是同分布评测，不含迁移。

V-STaR 在这一段里提出了一个后来变得很重要的观察：被丢弃的错误解本身是信号 [c-vstar-waste]。这句话是后面拒绝采样与过程奖励的前提。

4.3 第二段：模型当裁判

2024 年起改进对象上移到评判器：模型用 LLM-as-a-Judge 给自己发奖励，不再依赖独立的奖励模型 [c-evaluator-internalized]。Self-Rewarding 报告两条轴同时上升——指令跟随能力，以及给自己发高质量奖励的能力 [c-self-reward-double-axis]。

双轴主张是整个内化路线的前提。若评判能力不涨，第二条轴会先饱和——而 Meta-Rewarding 报告的正是这件事：只改答案不改判断会在迭代中迅速饱和；加上「评判自己的判断」后，Llama-3-8B-Instruct 在 AlpacaEval 2 上的胜率从 22.9% 升到 39.4% [c-meta-reward-saturation]。这是这条路线第一次自我否定，而且是否定自己上一代。

评判器自身也可以被完全自举：Self-Taught Evaluators 不用任何偏好标注，把 Llama3-70B-Instruct 在 RewardBench 上从 75.4 提到 88.3 [c-evaluator-self-taught]。

4.4 第三段：连题都不用出

2025 年起出现零外部数据自举：模型自己出题、自己解题，课程由「任务接近当前能力边界」这一条件生成，而不是由人编写题库 [c-zero-data]。动机写得直白——人类出的题，对未来超越人类的系统学习价值有限 [c-zero-data-incentive]。

SSR 走得更远：连自然语言的问题描述都去掉，用测试补丁形式化任务，在两个 SWE 基准上自改进 10.4 与 7.8 个点 [c-ssr-task-formalization]。

这一波里最直接把「改自己」当作动作的是 SEAL：模型生成自编辑（改写数据、指定超参、调用工具），再用更新后模型的下游表现作为奖励 [c-seal-weight-edit]。它把权重更新本身放进了策略的动作空间，代价是每一次动作都要真跑一次微调才能知道好坏——这是五层里最贵的一次评估。

出题机制也有三条互不相同的路子：与工具交互后出题并自带验证函数（Self-Challenging）、好奇心驱动的自提问（AgentEvolver）、只保留成功轨迹的经验库（SiriuS），而难度接近能力边界时学习效用最高（HS-STaR）[c-task-generation-mechanisms]。

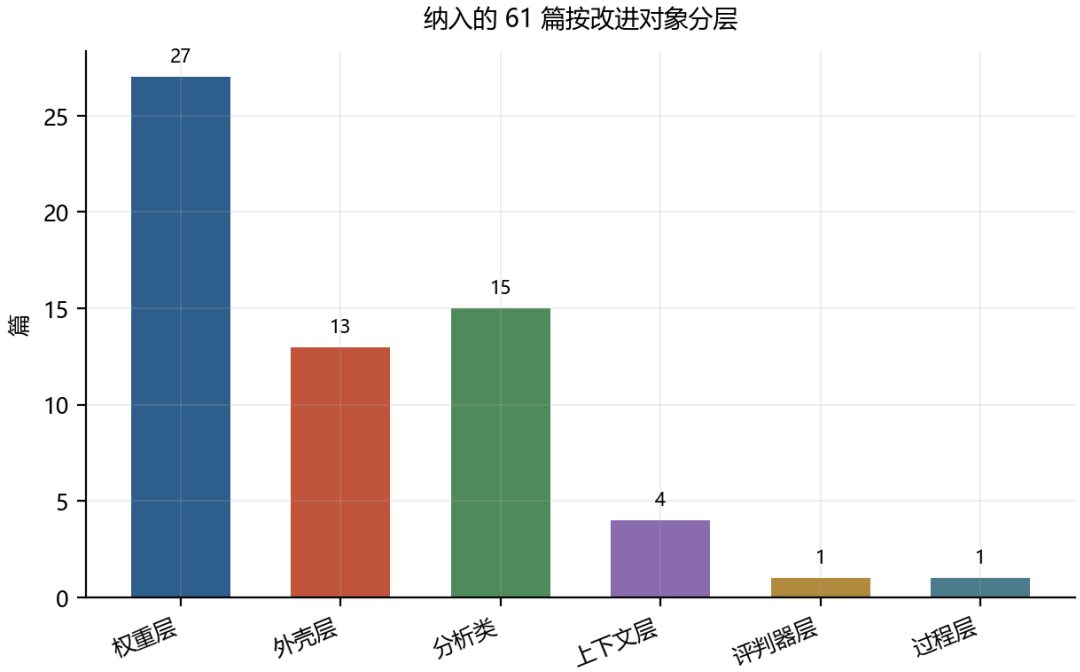


图 2　图 2　纳入的 61 篇按改进对象分层。权重层占近一半，说明「自我改进」在实际研究里主要指改参数；过程层只有 1 篇，说明最接近 RSI 字面意义的形态才刚出现。

还有一条非完全的自举：递归把难题分解成更简单的变体，LADDER 用这一招在本科水平题目上把 Llama 3.2 3B 的准确率从 1% 提到 82% [c-recursive-decomposition]。起点只有 1%，**本文推断**这个量级的相当一部分来自任务被改写而非模型变强——但摘要没有给出对应的对照，故这只是一个待验证的解释，不是结论。

更温和的一条是「用更好的自己当老师」：On-Policy Self-Training 的监督来自特权推理示范，即模型自己更会做的那一版，而不是外部教师 [c-privileged-distill]。这与 SEAL 同型，区别只在蒸馏还是改权重。

这里有一个必须说清的区分：**零数据不等于零监督**。锚还在，只是从「人出的题」换成了「环境反馈」——测试能不能通过。所以这一段真正做的是把监督外包给环境，而不是取消监督。这个区分在讨论「RSI 能否脱离人类」时是关键：环境是人造的。

4.5 第四段：用统计量代替判断

LaSeR 的结论最强：一个解的推理奖励等于它末 token 的自奖励分数，即对某个指定 token 的对数概率差 [c-laser-proxy]。这是锚点内移的极限形态——不再问「这个解好不好」，只问「这个模型对这个解有多确信」。

Auxiliary Rewards 用熵的全局变化与局部平滑度替代答案级奖励，在不改 TTRL 的前提下把在线宏平均提高 2.08 个百分点 [c-aux-reward]。

还有一种做法是往回走半步：RLSVR 不取消验证，而是把开放任务改造成可自验证的代理环境，用环境内部规则自动产生奖励 [c-rlsvr]。这与「零数据」不矛盾——锚点从人写的题换成了人造环境，而环境仍由人设计。

代价必须写清楚：**统计量与「好」之间的关系是经验性的，不是定义性的**。答案对错是定义，置信度高低是相关。越往后搬，奖励越容易与目标脱钩，而检测脱钩的手段恰恰是最薄的一环。

四个阶段与它们各自的锚见表 2。

5 冻结模型那一支

5.1 不是同一件事

外壳层的工作在机制上与权重层完全不同——它们是「冻结模型、只改外围代码」这一整类，共同点是不需要任何梯度 [c-harness-class]：ADAS 用元 agent 写代码发现新 agent，AlphaEvolve 用一个或多个评估器给每次改写背书，而 DGM 的自指性在于它改的是「自己改代码的能力」——代码变更提升的是后续变更的成功率，而不只是任务分数 [c-dgm-self-reference]。

SICA 是这一支最干净的展示：不带任何梯度，一个带基本编码工具的 agent 自主编辑自身代码，在 SWE-bench Verified 的随机子集上把成绩从 17% 提到 53% [c-sica-no-gradient]。这是单 benchmark 子集的小样本结果，数值只应读作「量级」。

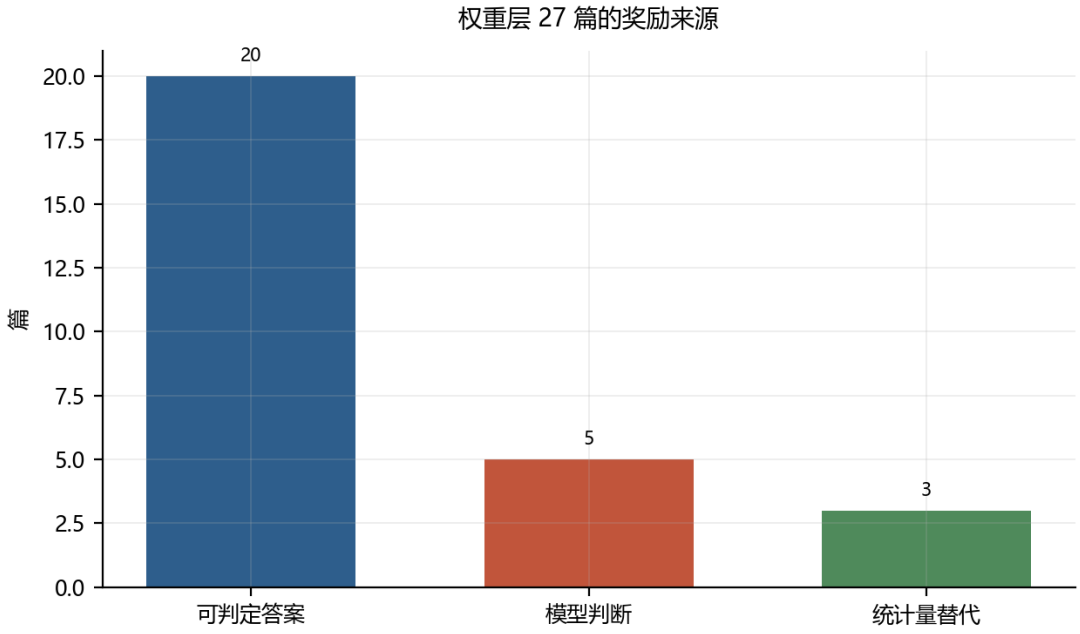


图 3　图 3　权重层 27 篇的奖励来源。可判定答案仍是主流（20 篇）；模型判断 5 篇、统计量 3 篇，两者都出现在「答案判不了」的开放任务上。

表 2　四个阶段与它们各自的锚。阶段划分按「奖励从哪来」，不按时间硬切。

阶段	时间	奖励从哪来	代表工作
人写答案	2022-2024	标准答案与人工偏好	STaR / ReST / SPIN / V-STaR
模型当裁判	2024-2025	LLM-as-a-Judge 自评	Meta-Rewarding / Self-Taught Evaluators
零数据自举	2025-2026	环境反馈（测试能否通过）	Absolute Zero / R-Zero / SSR
统计量替代	2025-2026	置信度与熵	LaSeR / Auxiliary Rewards

这类工作常与权重路线混为一谈，因为两者都自称 self-improving。但把它们混在一起会掩盖最关键的区别：**外壳可以回滚，权重不能。**

5.2 代价是样本效率

ShinkaEvolve 指出这类方法的公认瓶颈：既有代码演化方法需要上千个样本才能找到一个有效解 `[c-shinka-efficiency]`。换言之，外壳路线的「不需要梯度」是用「需要很多次昂贵评估」换来的。

这笔账在论文里常被略过，因为评估成本由检索与执行承担，不显示在训练曲线上。

5.3 上下文层：被讨论忽略的那一层

Reflexion 明确不走权重路线，只用语言反馈；Voyager 把经验固化成可执行代码的技能库；AWM 把复用的工作流抽成记忆，在 Mind2Web 与 WebArena 上相对成功率提升 24.6% 与 51.1% `[c-context-layer]`。

这一层常被排除在 RSI 之外，理由是它不产生持久的能力改变。但它恰恰是工业界实际在用的那一层，增益量级也不小。把它排除会让「RSI 离落地还有多远」这个问题的答案偏悲观——因为它已经在落地了，只是不叫这个名字。

6 失效模式

6.1 信号侧：判断者与被判断者同源

模型会系统性高估「高置信度的错误」，形成自我确认循环 `[c-self-confirming-loop]`。原文把它量化成三个量：奖励噪声、策略-奖励耦合、过度/不足奖励偏斜。

这不是调参问题。奖励由模型给出，而模型对错误的置信度与对正确的置信度同源，所以偏差随耦合强度放大。判据饱和是同一个洞的另一种表现：一个是偏置，一个是天花板。

六类失效与它们所在的层见表 3。

6.2 训练侧：熵与振荡

标准 RLVR 提升 pass@1 的代价是策略熵下降 `[c-entropy-tradeoff]`。多轮 agent 强化学习还会出现 Echo Trap：奖励方差出现悬崖、梯度出现尖峰 `[c-echo-trap]`。

还有一层更隐蔽的噪声：一条回答的奖励表示依赖它被随机分到的那一组上下文，因此单次分组实

表 3 六类失效与它们所在的层。「是否结构」一列区分「调参能修」与「改了结构才会变」。

失效	出现在哪一层	表现	是否结构
自我确认循环	评判器 / 权重	高置信度的错误被系统性高估	是（判断者与被判断者同源）
判据饱和	评判器 / 权重	只改答案不改判断时迅速饱和	是（同上）
Echo Trap	权重	奖励方差悬崖与梯度尖峰	未定
熵耗尽	权重	pass@1 上升而策略熵下降	是（可验证奖励的固有交换）
模型坍缩	权重	分布尾部不可逆损失	是（替换真实数据时）
验收门失明	外壳	重叠防护被逐个删掉而无观测伤害	是（门只看得见分数）
无停机判据	外壳 / 过程	停止与模型身份相关而非任务	是（缺任务侧终点）

现会给出不可靠的优势估计 [c-gmae-context]。在组相对的方法里，这意味着同一个回答在不同 batch 里可能得到不同的优势——它不是数据问题，是估计量问题。

熵这条证据决定了报告口径：只报 pass@1 的方法，无法区分「能力提升」与「多样性被榨干」——两者在 pass@1 上的表现完全一样。

6.3 分布侧：坍缩及其边界条件

在自己的输出上训练会造成不可逆的尾部损失（模型坍缩）[c-collapse]。规避条件是把各代合成数据与原始真实数据**累积**而非替换 [c-collapse-avoidable]。

但累积不是万能的：采样温度低于 1 时，累积也止不住谱隙收窄。两条证据方向相反却并存，这恰恰是这一节最该保留的形状——**「坍缩可避免」是一个有条件的结论，条件写在第二篇里。**

6.4 目标侧：验收门看不见自己在拆防护

这是本文读到的最不寻常的一条。当多个防护重叠时，删掉其中一个不会产生可观测伤害（被其他防护掩盖），于是验收门只按任务成功率决策，冗余在无人察觉中流失 [c-masked-erosion]。

这条失效的特殊性在于：它不是「奖励被 hack」，而是「验收门本身失去了分辨力」。而且它只出现在外壳/代码自改写这一类——因为只有这一类的动作空间里包含「修改防护」。所有基于「用分数当门」的自改进系统都有这个风险，而分数是最常用的门。

6.5 停止条件

自我修改何时停止与模型身份强相关，而与任务无关：同一套流程下，有的模型 100% 声明收敛，有的跑到 101 轮 [c-stop-identity]。

这说明「停止」目前不是任务侧的性质，而是模型侧的性质。对无人值守的闭环来说这是一个未解决的工程问题：没有一个与任务挂钩的停机判据。

7 证据强度

7.1 最硬的一条反证

在 pass@k 取大 k 时，基座模型的分数反而高于 RLVR 训练后的模型，且现有训练没有引出新的推理模式 [c-no-new-capability]。

这条证据的分量在于方法：跨多个模型家族、多个 RL 算法、数学/代码/视觉三类基准，用大 k 的 pass@k 作为能力边界指标。它不是说 RL 没用，而是说 **RL 把已有能力重新加权，没有扩张边界**。而「自我改进」这个词承诺的正是扩张。

7.2 审计给出的比例

在过去 90 篇自称 RSI 的研究中，只有 22% 审计了两次以上的完整循环，60% 在未用于选择的任务上评测，而 90% 报告了产品侧评测 [c-audit-gap]（图 4）。

分母必须说清楚：22% 的分母是那 90 篇**自称** RSI 的研究，不是全部 114 篇，更不是所有自改进工作。这个比例的含义是「声明与证据之间的落差有多大」，不是「RSI 不成立」。

7.3 理论上它应该有效

自奖励迭代对初始模型的依赖随迭代次数指数衰减，有限样本误差以 $O(1/\sqrt{n})$ 收敛 [c-theory-decay]。

这条与 §7.1 并不矛盾：理论说的是「在模型类与奖励都正确的条件下」，实证说的是「在当前模型与当前奖励下，边界没动」。两者放在一起读，指向同一个位置——**限制不在迭代次数，在锚的质量。**

7.4 报告口径的偏差

61 篇的主指标分布是：准确率 21、成功率 8、pass@1 5、胜率 5、pass@k 2、速度 2，另有 18 篇摘要不报单一主指标 [c-metric-gap]（图 5）。

pass@k 只有 2 篇。这不是巧合：**被最少报告的指标，正好是最难看的那个。**

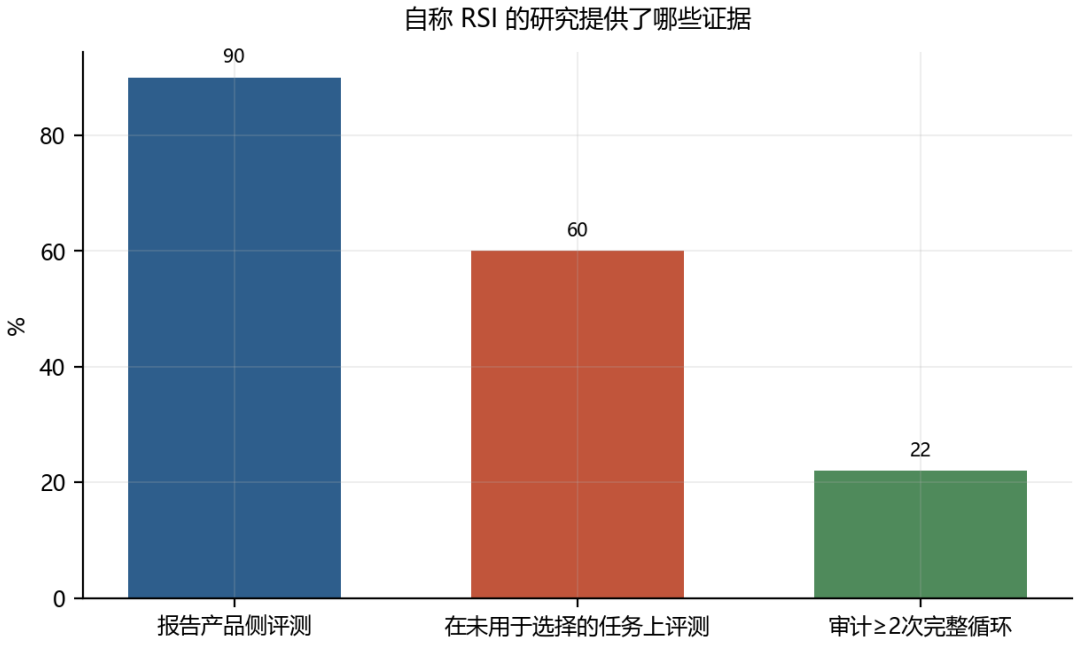


图 4　图 4　90 篇自称 RSI 的研究中，三类证据的覆盖率。60% 与 22% 之间的落差是本文最看重的一个数：在没被选择用过的任务上评测，与把整条循环重跑一遍，是两种完全不同强度的证据。

8 过程层：最新的形态

RSI-Opt 让 meta-agent 依据历史运行证据改写「指导训练优化的策略」，在 5 代演化中让 meta-agent 自己做 CPU 代理实验；Recursa 用并行学习器执行任务、合成局部改进、验证后合并进知识仓库，在留出任务上把全通过率从 30.0% 提到 55.0% [c-process-layer]。

这一层最接近 RSI 的字面意义：改进的对象包含改进机制本身。它也是最难验证的一层——你无法用一次实验区分「机制变好了」与「这次的任务恰好合适」。

RSI Arena 从另一侧给出可行性证据：十个前沿模型端到端做后训练，各自选数据、实现训练流程、选检查点，十个全部产出了可用助手 [c-rsi-arena]。「可用」由 20 个基准与盲评共同判定，不等于达到人类调优水平。

9 建议

9.1 按层决定放开程度

依据是表 1 的两个性质（可验证性、回滚成本），加上两个可测量的前置条件：验证器强度（与可信参照的一致度）与可恢复性（撤销一次坏改动的代价）[c-verifier-recoverability]。

最后一行的理由值得单独说：当被改进的对象包含改进机制本身时，之前所有的验证结论都要作废。这不是保守，是逻辑——你验证的是旧机制。

9.2 报告建议

1. **同时报 pass@1 与 pass@k。**只报前者无法区分能力提升与熵耗尽 [c-entropy-tradeoff]。
2. **报「未用于选择的任务」上的结果。** 60% 已经做到，这个数字应该变成 100% [c-audit-gap]。
3. **报完整循环的重复次数。**单次跑通不是 RSI 的证据，22% 说明这仍是少数派做法。
4. **报奖励来源。** judge 与 proxy 的失效模式与 answer 完全不同，混在一起报会掩盖风险 [c-anchor-migration]。

10 效度威胁与局限

10.1 全部证据为摘要级

本档 63 条引文全部是 arXiv 摘要的原文子串，未抓全文。因此凡是「是否做了消融」「表里的具体数值」这类问题，本文一律不下断言。

这是从同类工作的失败里学来的：当证据源比研究对象窄时，缺失会被误读为不存在。摘要是作者用来推销结论的，不是用来汇报实验设计的——用摘要的沉默去论证文献的空白，等于把自己的取材边界写成别人的缺陷。

10.2 61 篇是编码样本，不是全集

候选池 1006 篇，人工编码 61 篇。任何百分比都应读作「有这个现象」，不是「比例如此」。

同一件事还有另一个版本，值得写在这里：首轮按记忆列出的 66 个经典 ID 里 11 个是错的，而所有

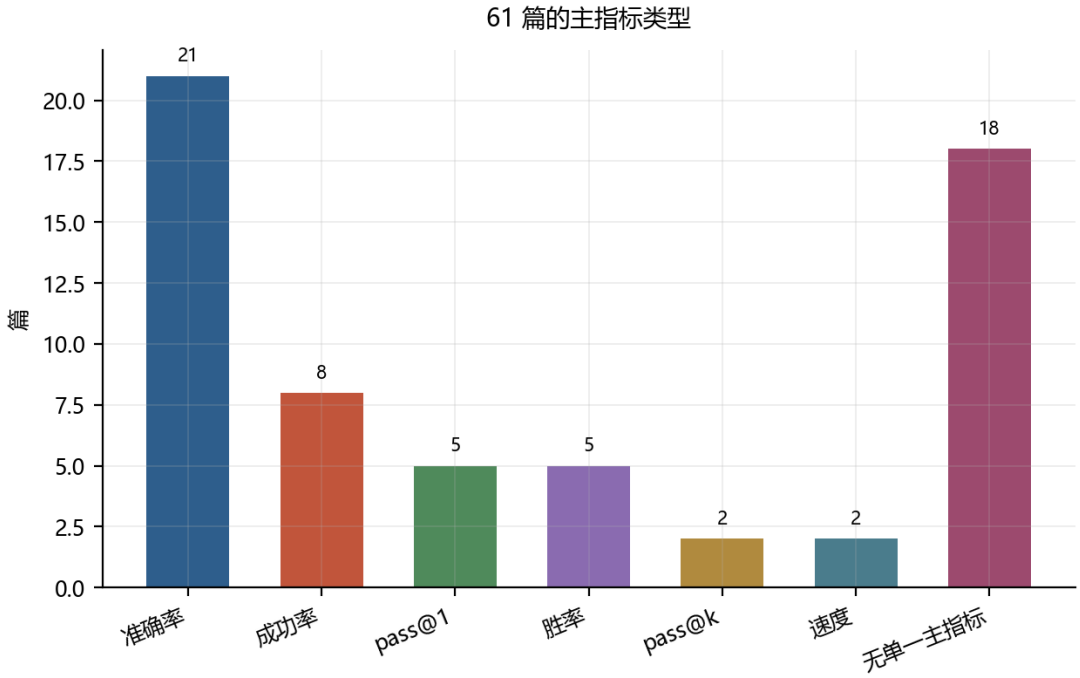


图 5　图 5　61 篇按摘要中的主指标归类。pass@k 只有 2 篇，而它是唯一能回答「能力边界有没有移动」的指标；速度只有 2 篇，说明成本几乎不在报告范围内。

表 4　分层放开建议。依据是回滚成本、可验证性，以及验证器强度与可恢复性两个可测前置条件。

层	是否可无人值守	理由
上下文	可以	回滚是删记录，验收可直接测任务成功率
外壳	有条件	回滚便宜，但验收门会掩盖防护流失
评判器	需抽检	判断者自举引入自我确认偏差，需外部参照集
权重	需人工签署	回滚成本高、单次更新不可归因，且坍缩不可逆
过程	仅限有档案时	改进机制被改后，此前验证结论全部失效

自动校验器都没报错——它们检查引用是否闭合，不检查引用是否指向你以为的那篇论文。**记忆里的 ID 与文件里的 ID 是两回事，只有逐条核对才能分开。**

10.3 口径可争议

分层与奖励来源都是本文的构造。换个口径（例如把 Self-Rewarding 归到评判器层）会得到另一组数。故所有统计都写在 `corpus_labels.csv` 里，可由 `analyze.py` 重算。

10.4 未覆盖物理 AI

检索里出现了大量机器人策略的 RSI 工作，它们的证据是物理实验，与文本基准不可直接比较，故排除。这是一个真实的边界：本文的结论对具身场景未必成立。

11 结论

四年里，模型确实学会了改进自己——但方式与「RSI」这个词暗示的不同。

变的是判断权的位置，不是算法。 61 篇里没有一篇提出新的优化目标；变化集中在「什么算好」这个判断由谁做出：人写答案、人写测试、模型当裁判、最后用一个标量代替裁判 [c-anchor-migration]。每一次内移都换来可扩展性，也都换来一类新的失效 [c-self-confirming-loop][c-masked-erosion]。

分层比排序更有用。与其问「哪个方法更强」，不如问「它在哪一层、失败能不能回滚、改动能不能验证」。这两个性质同向，于是可以直接推出哪一层能放开自动改、哪一层必须人工签署 [c-verifier-recoverability]。

证据强度是这条路线真正的短板。只有 22% 的自称 RSI 研究审计了两次以上完整循环，唯一能测能力边界是否移动的 pass@k 只有 2/61 篇报告 [c-audit-gap][c-metric-gap]。而最强的反证恰恰来自这个指标：大 k 时基座模型反而更高 [c-no-new-capability]。

如果要给一句话：**这个方向已经证明系统可以改进自己，但还没有证明改进的是边界，而不是权重。**

12 附录 A　可复现性

- 检索：`fetch_rsi.py`，16 组 arXiv 检索式取并集，候选池 1006 篇。
- 编码：`codebook.py`，61 篇 × 3 维标签，写入 `corpus_labels.csv`。
- 证据：`build_archive.py` 生成 63 条，**在写 CSV 之前**逐条校验是否为摘要原文子串，不是就退出、不写文件。
- 统计：`analyze.py` 生成 `analysis.json`，正文与图表的数字均引自该文件。
- 档案校验：`scripts/validate_research.py`。
- 论文校验：`scripts/audit_paper.py`（数值可回溯）、`scripts/check_paper.py`（内部自洽）。

参考文献

下列 53 个来源被正文实际引用；归档共 53 个（`sources.csv`），即归档全部。每条标题后的方括号是它支撑的主张编号。

[1] Recursive Self-Improvement in AI: From Bounded Self-Refinement to Autonomous Research Loops `arXiv:2607.07663` [c-corpus-composition] [c-corpus-timeline] [c-rsi-definition] [c-rsi-vocabulary]

[2] Trustworthy Recursive Self-Improvement in the Era of Large Models: Foundation, Risks, Evaluation and Open Challenges `arXiv:2610.35193` [c-rsi-definition]

[3] Darwin Godel Machine: Open-Ended Evolution of Self-Improving Agents `arXiv:2505.22954` [c-dgm-self-reference] [c-harness-class] [c-loop-layers] [c-rsi-definition]

[4] Self-Rewarding Language Models `arXiv:2401.10020` [c-anchor-migration] [c-evaluator-internalized] [c-self-reward-double-axis]

[5] LaSeR: Reinforcement Learning with Last-Token Self-Rewarding `arXiv:2510.14943` [c-anchor-migration] [c-laser-proxy]

[6] Self-Improvement with Auxiliary Rewards: Learning from Verified Traces and Evaluating Transfer `arXiv:2610.41296` [c-anchor-migration] [c-aux-reward]

[7] Absolute Zero: Reinforced Self-play Reasoning with Zero Data `arXiv:2505.03335` [c-anchor-migration] [c-zero-data] [c-zero-data-incentive]

[8] Automated Design of Agentic Systems `arXiv:2408.08435` [c-harness-class] [c-loop-layers]

[9] Self-Adapting Language Models `arXiv:2506.10943` [c-loop-layers] [c-seal-weight-edit]

[10] RSI-Opt: Recursive Self-Improvement Optimizer Agents For Model Training `arXiv:2610.32026` [c-loop-layers] [c-process-layer]

[11] A Survey on LLM Inference-Time Self-Improvement `arXiv:2412.14352` [c-survey-framing]

[12] A Survey of Self-Evolving Agents: What, When, How, and Where to Evolve on the Path to Artificial Super Intelligence `arXiv:2507.21046` [c-survey-framing]

[13] STaR: Bootstrapping Reasoning With Reasoning `arXiv:2203.14465` [c-star-loop] [c-star-scale]

[14] Reinforced Self-Training (ReST) for Language Modeling `arXiv:2308.08998` [c-rest-offline]

[15] Self-Play Fine-Tuning Converts Weak Language Models to Strong Language Models `arXiv:2401.01335` [c-spin-anchor]

[16] Quiet-STaR: Language Models Can Teach Themselves to Think Before Speaking `arXiv:2403.09629` [c-quiet-star-anchor]

[17] Iterative Reasoning Preference Optimization `arXiv:2404.19733` [c-iterative-preference]

[18] V-STaR: Training Verifiers for Self-Taught Reasoners `arXiv:2402.06457` [c-vstar-waste]

[19] Meta-Rewarding Language Models: Self-Improving Alignment with LLM-as-a-Meta-Judge `arXiv:2407.19594` [c-meta-reward-saturation]

[20] Self-Taught Evaluators `arXiv:2408.02666` [c-evaluator-self-taught]

[21] R-Zero: Self-Evolving Reasoning LLM from Zero Data `arXiv:2508.05004` [c-zero-data]

[22] Agent0: Unleashing Self-Evolving Agents from Zero Data via Tool-Integrated Reasoning `arXiv:2511.16043` [c-zero-data]

[23] Toward Training Superintelligent Software Agents through Self-Play SWE-RL `arXiv:2512.18552` [c-ssr-task-formalization]

[24] Self-Challenging Language Model Agents `arXiv:2506.01716` [c-task-generation-mechanisms]

[25] AgentEvolver: Towards Efficient Self-Evolving Agent System `arXiv:2511.10395` [c-task-generation-mechanisms]

[26] SiriuS: Self-improving Multi-agent Systems via Bootstrapped Reasoning `arXiv:2502.04780` [c-task-generation-mechanisms]

[27] HS-STaR: Hierarchical Sampling for Self-Taught Reasoners via Difficulty Estimation and Budget Reallocation `arXiv:2505.19866` [c-task-generation-mechanisms]

[28] LADDER: Self-Improving LLMs Through Recursive Problem Decomposition `arXiv:2503.00735` [c-recursive-decomposition]

[29] On-Policy Self-Training: Data-Free Self-Improvement in Large Language Models `arXiv:2610.32770` [c-privileged-distill]

[30] From RLVR to RLSVR: Task Transformation Induces Self-Verifiable Rewards for Open-Ended LLM Self-Improvement `arXiv:2607.23802` [c-rlsvr]

[31] Gödel Agent: A Self-Referential Agent Framework for Recursive Self-Improvement arXiv:2410.04444 [c-harness-class]

[32] AlphaEvolve: A coding agent for scientific and algorithmic discovery arXiv:2506.13131 [c-harness-class]

[33] DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines arXiv:2310.03714 [c-harness-class]

[34] A Self-Improving Coding Agent arXiv:2504.15228 [c-sica-no-gradient]

[35] ShinkaEvolve: Towards Open-Ended And Sample-Efficient Program Evolution arXiv:2509.19349 [c-shinka-efficiency]

[36] Reflexion: Language Agents with Verbal Reinforcement Learning arXiv:2303.11366 [c-context-layer]

[37] Voyager: An Open-Ended Embodied Agent with Large Language Models arXiv:2305.16291 [c-context-layer]

[38] Agent Workflow Memory arXiv:2409.07429 [c-context-layer]

[39] Breaking the Self-Confirming Loop: Diagnosing and Mitigating Systemic Reward Bias in Self-Rewarding RL arXiv:2510.08977 [c-self-confirming-loop]

[40] Beyond Pass@1: Self-Play with Variational Problem Synthesis Sustains RLVR arXiv:2508.14029 [c-entropy-tradeoff] [c-metric-gap]

[41] RAGEN: Understanding Self-Evolution in LLM Agents via Multi-Turn Reinforcement Learning arXiv:2504.20073 [c-echo-trap]

[42] Group-Marginalized Self-Rewarding RL Drives Zero-Label Self-Evolving arXiv:2609.36750 [c-gmae-context]

[43] The Curse of Recursion: Training on Generated Data Makes Models Forget arXiv:2305.17493 [c-collapse]

[44] Is Model Collapse Inevitable? Breaking the Curse of Recursion by Accumulating Real and Synthetic Data arXiv:2404.01413 [c-collapse-avoidable]

[45] Mitigating Model Collapse through Dreaming Learning arXiv:2610.30009 [c-collapse-avoidable]

[46] Masked Erosion: How Acceptance Gates Lose Redundant Safeguards in Self-Improving Agents arXiv:2610.38392 [c-masked-erosion]

[47] When LLMs Modify Themselves: Convergence, Values, and Stopping in Iterative Self-Improvement arXiv:2611.01304 [c-stop-identity]

[48] Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model? arXiv:2504.13837 [c-no-new-capability]

[49] Auditing the Product, Not the Recursion: The Evaluation Evidence Behind Recursive Self-Improvement Claims arXiv:2610.27610 [c-audit-gap] [c-metric-gap]

[50] Why Self-Rewarding Works: Theoretical Guarantees for Iterative Alignment of Language Models arXiv:2601.22513 [c-theory-decay]

[51] Recursa: Toward Recursively Self-Improving Foundation Model Systems arXiv:2611.04035 [c-process-layer]

[52] RSI Arena: Towards Recursive Self-Improvement with Human Feedback arXiv:2610.33624 [c-rsi-arena]

[53] Recoverability and Verifier Strength: Two Measured Preconditions for Unattended Agent Self-Improvement arXiv:2610.27465 [c-verifier-recoverability]